



طراحی سامانه‌های میکروسرویس

نیم‌سال دوم ۱۴۰۴-۰۵

استاد: دکتر علی ابریشمی - سیدحسین میرحسینی

دانشکده‌ی مهندسی کامپیوتر

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

تمرین پنجم

فهرست مسائل

۱	مسئله ۱
۱	الف) External Network
۲	ب) Linting
۲	ج) Image Registry
۳	مسئله ۲
۵	مسئله ۳
۷	مسئله ۴
۱۰	مسئله ۵
۱۴	مسئله ۶
۱۸	مسئله ۷
۲۱	مسئله ۸
۲۱	الف) بالا آوردن نکسوس
۲۱	ب) مراحل انجام‌شده
۲۲	ج) دریافت وابستگی‌ها از مخزن ساخته‌شده
۲۲	د) پوش و پول ایمیج
۲۳	ه) نقش‌های نکسوس در یک پایپلاین CI/CD
۲۴	و) کاربرد مخزن نوع Group

پاسخ مسئله‌ی ۱. تعریف مفاهیم

الف) External Network

وقتی با docker compose یک پروژه را بالا می‌آوریم، داکر به‌صورت پیش‌فرض برای همان پروژه یک شبکه‌ی اختصاصی می‌سازد و با پایین آوردن پروژه آن را حذف می‌کند. شبکه‌ی external یعنی شبکه‌ای که بیرون از این چرخه‌ی عمر ساخته شده است؛ ما فقط اعلام می‌کنیم که از قبل وجود دارد و داکر باید سرویس را به آن وصل کند، نه اینکه آن را بسازد یا پاک کند.

```

1 # docker network create network-name
2 networks:
3   network-name:
4     external: true

```

کاربرد اصلی‌اش این است که چند پروژه‌ی compose مستقل بتوانند همدیگر را با نام سرویس صدا بزنند، بدون اینکه مجبور باشیم همه را در یک فایل واحد جمع کنیم. نکته‌ی مهم این است که چون داکر کامپوز فعلی مالک آن شبکه نیست، اگر شبکه از قبل ساخته نشده باشد کامپوز خطا می‌دهد و پاک کردن آن هم وظیفه‌ی خود ماست.

ب) Linting

لینتینگ یعنی تحلیل ایستای کد؛ یعنی بررسی کد بدون اجرای آن برای پیدا کردن خطاهای احتمالی، الگوهای مشکوک و ناسازگاری با قواعد سبک تیم. تفاوتش با کامپایلر این است که کامپایلر فقط می‌پرسد آیا این کد معتبر است؟ ولی لینتر می‌پرسد آیا این کد خوب است؟: متغیر استفاده نشده، خطای نادیده گرفته شده، ایمپورت اضافی، نام‌گذاری خلاف قرارداد، پیچیدگی بیش از حد یک تابع و مواردی از این دست.

معمولا دو لایه دارد: formatter که فقط ظاهر کد را یکدست می‌کند (مثل gofmt یا prettier) و linter که به معنا و ساختار کد کار دارد (مثل golangci-lint یا ESLint). ارزش اصلی‌اش در تیم این است که بحث‌های سلیقه‌ای از code review حذف می‌شود و بازبین به جای شمردن فاصله‌ها، روی منطق تمرکز می‌کند. به همین دلیل لینت تقریبا همیشه اولین و ارزان‌ترین مرحله‌ی پایپلاین است.

البته بعضا مرحله فرمت و لینت را از هم جدا می‌کنند، اما بعضی تیم‌ها هم هر دو را در یک مرحله انجام می‌دهند.

ج) Image Registry

رجیستری، انبار نسخه‌بندی شده‌ی imageها است؛ همان نقشی که npm برای پکیج‌های جاوااسکریپت یا Maven Central برای جاوا دارد. ایمج با نام و تگ در آن ذخیره می‌شود، با docker push بالا می‌رود و با docker pull پایین می‌آید.

نکته‌ی فنی مهم این است که رجیستری ایمج را به صورت یک فایل یکپارچه نگه نمی‌دارد، بلکه یک manifest به علاوه‌ی لایه‌های ایمج را ذخیره می‌کند و هر لایه با digest شناخته می‌شود؛ بنابراین لایه‌های مشترک بین ایمج‌ها فقط یک بار ذخیره و منتقل می‌شوند. همین digest است که اجازه می‌دهد به جای تگ به image@sha256:... ارجاع دهیم.

پاسخ مسئله‌ی ۲. طراحی پایپلاین

در پروژه‌هایی که سرویس‌های مختلف شامل Frontend، Backend و تغییرات پایگاه داده در یک مخزن گیت مشترک نگهداری می‌شوند، طراحی پایپلاین CI/CD نیازمند جداسازی هوشمند مراحل، اجرای موازی برای بهینه‌سازی زمان، و ایجاد شرط‌های عبور محکم جهت کاهش ریسک استقرار است. پایپلاین پیشنهادی شامل ۶ مرحله اصلی به ترتیب زیر است:

۱. Lint و تحلیل ایستا

- مراحل اجرا:
 - اجرای لینترهای اختصاصی برای هر بخش (مانند ESLint برای Frontend و golangci-lint یا Flake8 برای Backend).
 - بررسی کیفیت کد، قالب‌بندی، خطاهای نحوی و صحت نگارش Database Migration ها.
 - شرط عبور: عدم وجود خطای Syntax یا شکست در قوانین تعریف‌شده (میزان Warnings قابل تنظیم است).
 - خروجی: گزارش Lint و تحلیل کیفیت کد.

۲. تست

- مراحل اجرا:
 - اجرای موازی Unit Tests برای Frontend و Backend.
 - اجرای Integration Tests با استفاده از یک پایگاه داده ایزوله.
 - بررسی صحت اجرای Database Migration ها و ارتباط صحیح Backend با پایگاه داده.
 - شرط عبور: موفقیت تمامی تست‌های واحد و یکپارچه‌سازی و دستیابی به حداقل پوشش کد تعیین‌شده.
 - خروجی: گزارش نتایج تست‌ها و گزارش پوشش کد.

۳. ساخت و انتشار ایميج

- مراحل اجرا:
 - کامپایل پروژه‌ها و ساخت ایميج‌های داکر برای Frontend و Backend.
 - برچسب‌گذاری ایميج‌ها بر اساس Commit SHA و نسخه.
 - اسکن امنیتی ایميج‌ها برای شناسایی آسیب‌پذیری‌ها.
 - Push ایميج‌ها به Container Registry.
 - شرط عبور: ساخت موفق ایميج‌ها و عدم وجود آسیب‌پذیری بحرانی در اسکن امنیتی.
 - خروجی: ایميج‌های داکر آماده استقرار در Registry.

۴. استقرار در محیط آزمایشی

- مراحل اجرا:
 - اجرای خودکار Database Migration روی محیط Staging.
 - به‌روزرسانی سرویس‌های Frontend و Backend.
 - اجرای Smoke Test و تست‌های End-to-End.
 - شرط عبور: استقرار موفق بدون Downtime و موفقیت کامل تست‌های دود.
 - خروجی: محیط Staging به‌روز و آماده ارزیابی نهایی.

۵. تأیید دستی

- مراحل اجرا:

– بررسی نتایج تست‌ها، گزارش‌های امنیتی و وضعیت محیط Staging.

– تأیید انتشار توسط مسئول پروژه یا تیم عملیات.

• شرط عبور: دریافت تأییدیه انتشار.

• خروجی: مجوز آغاز استقرار در محیط عملیاتی.

۶. استقرار در محیط عملیاتی

• مراحل اجرا:

– اجرای Database Migration های سازگار با نسخه‌های قبلی.

– استقرار تدریجی با استفاده از Blue-Green Deployment یا Canary Deployment.

– بررسی Health Check ها و مانیتورینگ سرویس پس از انتشار.

– امکان بازگشت سریع در صورت مشاهده خطا.

• شرط عبور: سلامت کامل سرویس پس از استقرار و موفقیت تمامی Health Check ها.

• خروجی: نسخه جدید روی محیط عملیاتی به همراه مانیتورینگ و قابلیت بازگشت سریع.

تأیید دستی در دو نقطه کلیدی اضافه می‌شود:

۱. پیش از استقرار در محیط Production:

• دلیل: جلوگیری از انتشار ناخواسته کد به محیط اصلی، هماهنگی زمان انتشار با تیم‌های مارکتینگ/پشتیبانی، و انجام بررسی‌های نهایی توسط مدیر محصول یا لید فنی روی محیط Staging.

۲. پیش از اجرای Migration های حساس روی دیتابیس Production:

• دلیل: اسکرپت‌های تغییر ساختار دیتابیس ممکن است زمان‌بر یا سنگین باشند؛ لذا نیاز به بررسی و تأیید DBA یا ارشد تیم جهت جلوگیری از قفل شدن جدول‌ها دارد.

برای مدیریت ریسک و بازگشت سریع به حالت پایدار در صورت بروز خطا:

۱. بازگشت اپلیکیشن: از آنجا که تمام ایمیج‌ها بر اساس Commit SHA تگ خورده‌اند، در صورت شناسایی خطای بحرانی در Production، پایپلاین می‌تواند با یک کلیک یا به صورت خودکار، ترافیک را به تگ ایمیج نسخه قبلی هدایت کند.

۲. بازگشت پایگاه داده:

• اسکرپت‌های Migration باید همواره دارای متد Down باشند.

• تمام تغییرات دیتابیس باید بر اساس اصل Expand and Contract طراحی شوند تا نسخه جدید کدهای Backend با ساختار قبلی و نسخه جدید دیتابیس به طور هم‌زمان سازگار باشند.

پاسخ مسئله‌ی ۳. خرس

خواسته‌ی کارمی درست است: در تیمی که چند نفر هم‌زمان روی منو، سفارش و هماهنگی آشپزخانه کار می‌کنند، تنها جایی که می‌توان جلوی ورود مشکل به شاخه‌ی اصلی را گرفت، قبل از ادغام است. بعد از ادغام، هزینه‌ی پیدا کردن مقصر و برگرداندن تغییر چند برابر می‌شود.

تفاوت پایپلین merge request با پایپلین شاخه‌ی اصلی:

پایپلین main	پایپلین MR	
آیا آماده‌ی انتشار است؟	آیا اجازه‌ی ورود دارد؟	هدف
خود main	نتیجه‌ی ادغام شاخه با main	کد مورد آزمون
production و staging و سپس	هیچ (یا حداکثر review app موقت)	استقرار
پوش ایمیج با تگ نسخه	فقط بیلد، بدون پوش پایدار	انتشار artifact
کل مجموعه، شامل e2e کامل	تست‌های سریع + تست‌های متأثر	دامنه‌ی تست
هشدار فوری به تیم، مسدودسازی انتشار	جلوی merge را می‌گیرد	سیاست شکست

مهم‌ترین تفاوت فنی، همان سطر دوم است. اجرای پایپلین روی آخرین کامیت شاخه‌ی توسعه‌دهنده کافی نیست، چون ممکن است آن شاخه سبز باشد ولی بعد از ادغام با main خراب شود (semantic conflict: مثلاً یک نفر امضای تابع محاسبه‌ی قیمت سفارش را عوض کرده و نفر دیگر هم‌زمان یک فراخوانی جدید به همان تابع اضافه کرده است؛ گیت تعارض نمی‌بیند ولی کد کامپایل نمی‌شود). بنابراین پایپلین باید روی merge result اجرا شود و اگر main جلو رفت، دوباره اجرا شود.

مراحل پایپلین merge request:

۱. بررسی‌های ارزان: قالب پیام کامیت، نبود فایل حجیم، secret detection، لینت فقط روی فایل‌های تغییرکرده. این‌ها اول اجرا می‌شوند تا بازخورد تقریباً بلافاصله برسد.
۲. lint و بررسی تایپ: formatter، لینتر و کامپایل تایپ روی کل پروژه.
۳. unit test: سریع، بدون وابستگی خارجی، به همراه گزارش پوشش و مقایسه با main (کاهش پوشش، خودش یک هشدار است).
۴. build: بیلد ایمیج سرویس‌ها. اگر کد کامپایل شود ولی ایمیج ساخته نشود، مشکل واقعی است و باید همین‌جا دیده شود.
۵. integration test: با بالا آوردن وابستگی‌های واقعی (پایگاه داده، Redis، صف سفارش‌ها) به‌صورت سرویس کنار جاب؛ مسیرهای حساسی مثل ثبت سفارش و اعلان به آشپزخانه.
۶. بررسی امنیتی: SAST روی diff، اسکن وابستگی‌ها و اسکن ایمیج ساخته‌شده.
۷. review app: یک نمونه‌ی موقت از کل سامانه با URL یکتا برای هر MR، تا کارمی بتواند تغییر منو را واقعا ببیند نه اینکه تصورش کند. با بسته شدن MR خودکار حذف می‌شود. البته این مورد اختیاری هست و معمولاً استفاده نمی‌شود.

مسدودکننده (blocking):

- شکست کامپایل یا بیلد ایمیج
- حداقل یکی از تست‌های واحد یا یکپارچگی شکست‌خورده باشد
- خطای لینت و formatter

- آسیب‌پذیری high یا critical در وابستگی‌ها، و هر secret پیدا شده در کد
 - افت پوشش تست زیر آستانه‌ی توافق‌شده
 - غیرمسدودکننده (فقط هشدار، allow_failure):
 - آسیب‌پذیری low/medium بدون وصله‌ی موجود
 - هشدارهای کارایی مثل افزایش حجم bundle یا زمان بیلد
- معیار ساده: چیزی که قطعیت دارد و قابل رفع توسط نویسنده‌ی همین تغییر است، مسدودکننده باشد؛ چیزی که قضاوتی است یا رفعش به تصمیم تیم نیاز دارد، هشدار باشد.
- رساندن نتیجه به بازیبن‌ها و کارمی:
- وضعیت روی خود MR: هر جاب یک status check جداگانه با نام گویا؛ بازیبن در یک نگاه می‌فهمد کدام مرحله خراب است، نه اینکه فقط قرمز ببیند.
 - قوانین حفاظت از شاخه: دکمه‌ی merge تا سبز شدن بررسی‌های اجباری غیرفعال باشد. اتکا به یادت باشد قبل از merge نگاه کنی جواب نمی‌دهد.
 - گزارش‌های خلاصه به صورت کامنت خودکار: تست‌های شکست‌خورده با نام و پیام، اختلاف پوشش نسبت به main، و فهرست آسیب‌پذیری‌های تازه‌وارد شده همه در قالب diff، نه فهرست کامل؛ چون بازیبن فقط مسئول تغییر است.
 - artifact‌های قابل دانلود: گزارش پوشش، خروجی تست‌ها، لاگ کامل هر جاب و لاگ تست‌های e2e شکست‌خورده.
 - لینک review app مستقیم در توضیحات MR برای بررسی دستی توسط کارمی.
 - قوانین تأیید: حداقل یک تأیید از صاحب آن بخش (CODEOWNERS)؛ برای مثال تغییر در ماژول آشپزخانه نیاز به تأیید مسئول همان ماژول دارد.
- با این ساختار، تصمیم کارمی مبتنی بر شواهد است: پایپلاین می‌گوید چه چیزی خراب است، بازیبن می‌گوید آیا این راه‌حل درستی است، و دکمه‌ی merge تنها وقتی فعال می‌شود که هر دو پاسخ مثبت باشند.

پاسخ مسئله‌ی ۴. داکرایز

مراحل داکرایز کردن

مرحله اول: Builder Stage

- **FROM python:3.10-slim AS builder**: ایجاد یک مرحله مستقل با نام builder برای ایزوله کردن فرایند ساخت و کامپایل وابستگی‌ها.
- **ENV PYTHONDONTWRITEBYTECODE=1**: جلوگیری از تولید فایل‌های .pyc.
- **ENV PYTHONUNBUFFERED=1**: ارسال مستقیم خروجی‌های stdout و stderr به لاگ‌های داکر بدون buffer شدن.
- **ARG MIRROR_URL=""**: تعریف یک متغیر مخصوص زمان build برای تغییر منبع دانلود بسته‌ها.
- **RUN --mount=type=cache,target=/root/.cache/pip**: استفاده از قابلیت BuildKit Cache Mount برای کش کردن بسته‌های دانلودشده توسط pip و کاهش زمان بیلدهای بعدی.
- **-prefix=/install**: نصب وابستگی‌ها در مسیر ایزوله /install تا بتوان آن‌ها را به مرحله نهایی منتقل کرد.

مرحله دوم: Runner Stage (Final Image)

- **FROM python:3.10-slim AS runner**: شروع یک مرحله جدید و سبک بدون ابزارها و کش‌های مرحله ساخت.
- **PATH و PYTHONPATH**: افزودن مسیر بسته‌های نصب‌شده به متغیرهای محیطی.
- **RUN addgroup ... && adduser ...**: ایجاد یک کاربر Non-root برای افزایش امنیت و جلوگیری از حملات Container Breakout.
- **COPY --from=builder /install /install**: انتقال وابستگی‌های نصب‌شده بدون انتقال ابزارهای مرحله ساخت.
- **RUN chown -R appuser:appgroup /app**: واگذاری مالکیت فایل‌های پروژه به کاربر غیرریشه.
- **USER appuser**: اجرای برنامه با کاربر غیرریشه.

نحوه Build و اجرا

ساخت ایمیج با فعال بودن BuildKit:

```
1 DOCKER_BUILDKIT=1 docker build \
2   --build-arg MIRROR_URL=https://pypi.tuna.tsinghua.edu.cn/simple \
3   -t my-python-app:v1 .
```

اجرای کانتنر:

```
1 docker run --rm my-python-app:v1
```

نقش کلیدواژه‌ها

ENV متغیر محیطی تعریف می‌کند که هم در زمان build و هم در زمان اجرای کانتینر در دسترس است و در metadata ایمیج ذخیره می‌شود. بنابراین محل مناسبی برای نگهداری اطلاعات محرمانه نیست.

WORKDIR پوشه کاری دستورات بعدی مانند RUN، COPY و CMD را تعیین می‌کند و در صورت نیاز آن را ایجاد می‌کند.

AS به یک مرحله ساخت نام می‌دهد تا بتوان از آن در دستورهایی مانند COPY -from یا target- استفاده کرد.

ADD در برابر COPY

هر دو فایل‌ها را از build context به داخل ایمیج منتقل می‌کنند، اما ADD دو قابلیت اضافه دارد:

- استخراج خودکار آرشیوهای محلی؛
- دانلود فایل از یک URL (و در نسخه‌های جدید BuildKit، کلون کردن مخزن گیت).

به همین دلیل معمولاً استفاده از COPY توصیه می‌شود و فقط در صورت نیاز به استخراج خودکار آرشیو از ADD استفاده می‌شود.

EXPOSE در برابر Port-Mapping

8000 EXPOSE فقط اعلام می‌کند که برنامه روی این پورت گوش می‌دهد و هیچ پورتنی را منتشر نمی‌کند.

در مقابل، docker run -p 8000:8000 یا کلید ports در Docker Compose باعث ایجاد نگاشت واقعی پورت روی میزبان می‌شود.

ARG در برابر ENV

ENV	ARG	
زمان build و زمان اجرا	فقط زمان build	دامنه حیات
env_file یا مقدار پیش فرض -e	-build-arg	تعیین مقدار
بله، در metadata	خیر (اما در docker history)	حضور در ایمیج نهایی
بله	خیر	دیده شدن در برنامه
به مراحل بعدی ارث می‌رسد	فقط همان مرحله	دامنه اثر

البته که هیچ‌کدام برای نگهداری secret مناسب نیستند.

CMD در برابر ENTRYPOINT

ENTRYPOINT مشخص می‌کند کانتینر چه فرایندی را اجرا می‌کند و معمولاً ثابت در نظر گرفته می‌شود؛ CMD آرگومان‌های پیش فرضی است که به آن فرایند داده می‌شود و کاربر می‌تواند هنگام docker run آن را بازنویسی کند. وقتی هر دو با هم بیایند، دستور نهایی از الحاق این دو ساخته می‌شود:

```
1 ENTRYPOINT ["uvicorn"]
2 CMD ["app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- `docker run img ← docker run img -host 0.0.0.0 -port 8000` (چیزی داده نشده، پس CMD پیش فرض به ENTRYPOINT الحاق می‌شود).
- `docker run img -version ← docker run img -version` (آرگومان داده شده فقط جای CMD را می‌گیرد، ENTRYPOINT ثابت می‌ماند).
- تغییر خود ENTRYPOINT فقط با فلگ `docker run -entrypoint ...` امکان پذیر است.

اگر فقط CMD تعریف شود (بدون ENTRYPOINT)، هر آرگومانی که در `docker run` داده شود کل دستور را جایگزین می‌کند؛ این رفتار برای ایمیج‌های چندمنظوره مناسب است، ولی برای یک سرویس مشخص، ترکیب ENTRYPOINT به همراه CMD رفتاری ثابت تر و قابل پیش بینی تر می‌دهد.

نکته مهم دیگر این که هر دو باید به صورت `exec` (آرایه JSON، مانند مثال بالا) نوشته شوند، نه به صورت `shell`. در قالب `shell` (مثلاً `python main.py` CMD)، فرایند اصلی برنامه زیر `/bin/sh` اجرا می‌شود و خودش PID 1 نیست؛ در نتیجه سیگنال `SIGTERM` که `docker stop` می‌فرستد به آن نمی‌رسد و کانتینر به جای خاموشی تمیز، تا رسیدن `timeout` معطل می‌ماند و سپس با `SIGKILL` بسته می‌شود.

ساخت و اجرای کانتینر از روی Dockerfile

```

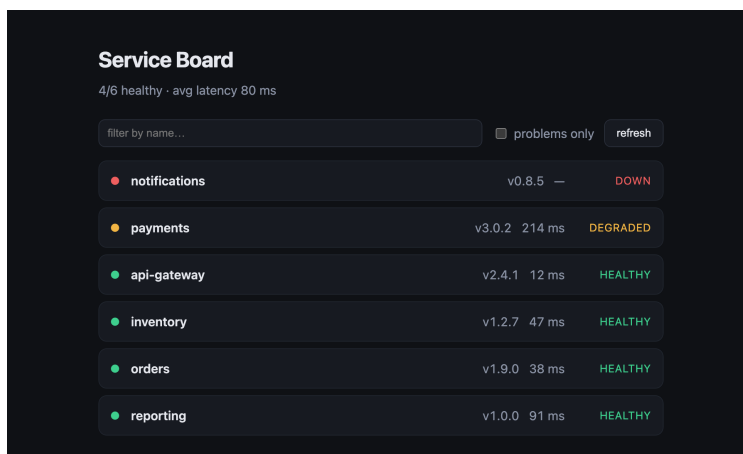
1 # build
2 docker build -t my-python-app:v1 .
3
4 # run
5 docker run --rm my-python-app:v1
6
7 # verify
8 docker run --rm my-python-app:v1 whoami
9
10 # check size and layers
11 docker image inspect my-python-app:v1 --format '{{.Size}}'
12 docker history my-python-app:v1

```

از آن جا که این برنامه یک اسکریپت یک بار مصرف است، نه یک سرویس شنونده روی پورت، نیازی به `-p`، `EXPOSE` یا `docker compose` ندارد؛ کانتینر پس از اجرای `main.py` و چاپ خروجی به صورت خودکار خاتمه می‌یابد و به لطف `rm` حذف هم می‌شود.

پاسخ مسئله‌ی ۵. تغییر

پروژه یک برنامه‌ی React 18 با TypeScript و Vite است (داشبورد وضعیت سرویس‌ها) و طبق صورت سؤال هیچ خطی از کد برنامه تغییر نکرده؛ تنها فایل‌های مرتبط با داکر اضافه یا اصلاح شده‌اند.



شکل ۱: برنامه‌ای که داکرایز شده است، در حال سرو شدن از ایمپج بهینه‌شده روی nginx.

مشکلات Dockerfile اولیه

```
1 FROM node:latest
2 WORKDIR /app
3 COPY . .
4 RUN npm install
5 RUN npm run build
6 CMD ["npm", "start"]
```

- node:latest هم غیرقابل تکرار است (فردا نسخه‌ی دیگری است) و هم حدود 1.1GB فقط پایه است.
- ایمپج نهایی شامل Node، npm، کل node_modules و سورس‌هاست، در حالی که خروجی واقعی فقط چند فایل ایستا در dist/ است.
- COPY . . قبل از npm install است؛ پس با تغییر یک کاراکتر در یک کامپوننت، لایه‌ی نصب وابستگی‌ها باطل و npm install از نو اجرا می‌شود.
- نبود dockerignore. یعنی node_modules میزبان هم وارد context می‌شود؛ روی macOS/arm64 این فایل‌های باینری برای ایمپج linux نامعتبرند.
- npm install به جای npm ci؛ ممکن است نسخه‌ها با lockfile یکی نباشند.
- برنامه با کاربر root و از طریق سرور توسعه‌ای vite preview سرو می‌شود، نه یک وب‌سرور واقعی.

نسخه‌ی بهینه‌شده

```
1 ARG NODE_VERSION=22.12.0
2 ARG NGINX_VERSION=1.27
3
4 FROM node:${NODE_VERSION}-alpine AS deps
```

```

5 WORKDIR /app
6 COPY package*.json ./
7 RUN --mount=type=cache,target=/root/.npm \
8     npm ci --no-audit --no-fund
9
10 FROM node:${NODE_VERSION}-alpine AS build
11 WORKDIR /app
12 COPY --from=deps /app/node_modules ./node_modules
13 COPY . .
14 RUN npm run build
15
16 FROM nginx:${NGINX_VERSION}-alpine AS runtime
17 COPY nginx.conf /etc/nginx/conf.d/default.conf
18 COPY --from=build /app/dist /usr/share/nginx/html
19 EXPOSE 80
20 HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
21     CMD wget --spider -q http://127.0.0.1/ || exit 1
22 CMD ["nginx", "-g", "daemon off;"]

```

مقایسه‌ی حجم و سرعت

حجم ایمیج	بیلد کامل	بیلد پس از یک خط تغییر	
1.34 GB (۱۲۷۴ مگابایت)	201 s	12.7 s	Dockerfile اولیه
49.9 MB (۵/۴۷ مگابایت)	137 s	1.9 s	Dockerfile بهینه
≈ ۲۷ برابر کوچک‌تر	≈ ۵/۱ برابر	≈ ۷/۶ برابر سریع‌تر	بهبود

```

q5 — image size + serving

$ docker images service-board
service-board:original 1.34GB
service-board:optimized 49.9MB

original : 1274.1 MB
optimized : 47.5 MB

$ docker compose up -d && curl -sI localhost:8088/
HTTP/1.1 200 OK
Server: nginx/1.27.5
Date: Thu, 06 Aug 2026 08:02:03 GMT
Content-Type: text/html
Content-Length: 398
Last-Modified: Thu, 06 Aug 2026 07:59:48 GMT
$ curl -s localhost:8088/ | head -c 200
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Service Board</title>
  <script type="m

$ curl -sI localhost:8088/assets/(...).js # immutable caching
asset: /assets/index-CB94t5Fy.js
Content-Type: application/javascript
Cache-Control: max-age=31536000
Cache-Control: public, immutable
$ curl -s -H "Accept-Encoding: gzip" -o /dev/null -w "gzip bytes: %{size_download}"
" localhost:8088$asset
gzip bytes: 54970
plain bytes: 145822

$ curl -s -o /dev/null -w "%{http_code}"
" localhost:8088/some/spa/route # SPA fallback
200

```

شکل ۲: حجم دو ایمیج و بررسی سرویس‌دهی ایمیج بهینه: gzip، کش تغییرناپذیر برای فایل‌های هش‌دار و fallback مخصوص SPA.

```

q5 — incremental rebuild

# incremental rebuild after a ONE LINE change in src/App.tsx

$ time docker build -f Dockerfile.original -t service-board:original .
#4 [1/5] FROM docker.io/library/node:latest@sha256:3e9ed202a2401a08687cb02367f7454a51c2a9e757d8e45728ec97a3c6241158
#6 [2/5] WORKDIR /app
#7 [3/5] COPY . .
#8 [4/5] RUN npm install
#9 [5/5] RUN npm run build
real 12.66

$ time docker build -t service-board:optimized .
#11 [deps 4/4] RUN --mount=type=cache,target=/root/.npm npm ci --no-audit --no-fund
#11 CACHED
#12 [deps 3/4] COPY package.json package-lock.json ./
#12 CACHED
#13 [build 3/5] COPY --from=deps /app/node_modules ./node_modules
#13 CACHED
#14 [build 4/5] COPY . .
#15 [build 5/5] RUN npm run build
#16 [runtime 2/3] COPY nginx.conf /etc/nginx/conf.d/default.conf
#16 CACHED
#17 [runtime 3/3] COPY --from=build /app/dist /usr/share/nginx/html
real 1.89

```

شکل ۳: بیلد مجدد پس از تغییر یک خط در src/App.tsx: در نسخه‌ی اولیه npm install دوباره اجرا می‌شود، در نسخه‌ی بهینه لایه‌ی وابستگی‌ها CACHED می‌ماند.

توضیح تغییراتی که باعث بهینه‌سازی شدند

۱. بیلد چندمرحله‌ای: بزرگ‌ترین عامل کاهش حجم، Node، npm و node_modules فقط در مراحل deps و build می‌مانند و به ایمیج نهایی راه پیدا نمی‌کنند؛ آنچه منتقل می‌شود تنها dist/ است (حدود ۱۵۰ کیلوبایت).
۲. تعویض پایه‌ی زمان اجرا: nginx:1.27-alpine به جای node:latest؛ برای فایل ایستا به Node نیازی نیست و nginx هم سریع‌تر و هم امن‌تر سرو می‌کند.
۳. تگ‌های پین‌شده: نتیجه‌ی بیلد امروز و سه ماه بعد یکی است.
۴. ترتیب لایه‌ها: package.json و package-lock.json جدا و قبل از سورس کپی می‌شوند؛ تا وقتی lockfile عوض نشود، لایه‌ی نصب از کش می‌آید. این تنها دلیل کاهش ۱۲.۷ ثانیه به ۱.۹ ثانیه است.
۵. npm ci + cache mount: npm ci دقیقاً از lockfile نصب می‌کند و mount=type=cache - کش npm را بین بیلدها نگه می‌دارد بدون اینکه داخل لایه‌ای ذخیره شود.
۶. .dockerignore: node_modules، dist و .git از context حذف شدند؛ هم ارسال context سبک شد و هم باینری‌های میزبان به ایمیج نشت نمی‌کنند.
۷. پیکربندی سرو: gzip، هدر Cache-Control: immutable برای مسیر /assets/ (چون Vite نام فایل‌ها را هاش دار می‌سازد)، no-cache برای index.html و try_files ... /index.html برای مسیریابی سمت کلاینت. اندازه‌ی باندل اصلی از ۱۴۵ KB به ۵۵ KB روی شبکه کاهش یافت.
۸. سخت‌سازی: HEALTHCHECK، فایل سیستم فقط‌خواندنی و no-new-privileges در کامپوز.

nginx.conf و docker-compose.yml

```

1 services:
2   web:
3     build:
4       context: .
5       dockerfile: Dockerfile
6       target: runtime
7     image: service-board:optimized
8     container_name: service-board
9     restart: unless-stopped
10    ports:
11      - "${WEB_PORT:-8088}:80"
12    healthcheck:
13      test: ["CMD", "wget", "--spider", "-q", "http://127.0.0.1/"]
14      interval: 10s
15      timeout: 3s
16      retries: 5
17    read_only: true
18    tmpfs:
19      - /var/cache/nginx
20      - /var/run
21    security_opt:
22      - no-new-privileges:true

```

```

1 server {
2   listen      80;
3   server_name _;
4   root        /usr/share/nginx/html;
5   index       index.html;
6
7   gzip on;   gzip_vary on;   gzip_min_length 1024;
8   gzip_types text/css text/plain application/javascript application/json image/
9             svg+xml;
10
11   location /assets/ {
12     expires      1y;
13     add_header   Cache-Control "public, immutable";
14     try_files    $uri =404;
15   }
16
17   location = /index.html { add_header Cache-Control "no-cache"; }
18
19   location / { try_files $uri $uri/ /index.html; }
20 }

```

پاسخ مسئله‌ی ۶. کامپوز

سه شبکه‌ی مجزا تعریف شده است. برنامه‌ی جاوا فقط در edge-net و redis-net است و helper فقط در helper-net؛ بنابراین این دو هیچ شبکه‌ی مشترکی ندارند و تنها مسیر ممکن از جاوا به helper، عبور از nginx است که تنها عضو مشترک edge-net و helper-net است. این‌گونه شرط انجینکس حتما باید بین برنامه‌ی جاوا و آن سرویس باشد با توپولوژی تضمین می‌شود، نه با قرارداد.

محدودیت	راه‌حل
فقط برنامه‌ی جاوا port mapping داشته باشد	کلید ports فقط زیر سرویس app آمده است
کلمه‌ی environment استفاده نشود	متغیرها از طریق env_file تزریق می‌شوند
nginx بین جاوا و سرویس کمکی	جداسازی شبکه‌ها
نیاز به Redis در زمان بالا آمدن	service_healthy depends_on با شرط

:docker-compose.yml

```

1 name: ${COMPOSE_PROJECT_NAME:-ms-hw5-compose}
2
3 services:
4   redis:
5     image: ${REDIS_IMAGE}
6     container_name: hw5-redis
7     command: ["redis-server", "--save", "60", "1", "--loglevel", "warning"]
8     volumes: [redis-data:/data]
9     networks: [redis-net]
10    healthcheck:
11      test: ["CMD", "redis-cli", "ping"]
12      interval: 5s
13      retries: 10
14
15    helper:
16      build: { context: ./helper, dockerfile: Dockerfile }
17      image: ${HELPER_IMAGE}
18      container_name: hw5-helper
19      networks: [helper-net] # no ports: reachable only from nginx
20      healthcheck:
21        test: ["CMD", "python", "-c",
22              "import urllib.request; urllib.request.urlopen('http://127.0.0.1:9090/
23              helper').read()"]
24        interval: 5s
25        retries: 10
26
27    nginx:
28      image: ${NGINX_IMAGE}
29      container_name: hw5-nginx
30      volumes: ["/./nginx.conf:/etc/nginx/nginx.conf:ro"]
31      networks: [edge-net, helper-net] # the only member of both networks
32      depends_on:
33        helper: { condition: service_healthy }
34      healthcheck:
35        test: ["CMD", "wget", "--spider", "-q", "http://127.0.0.1/healthz"]
36        interval: 5s
37        retries: 10
38
39    app:
40      image: ${APP_IMAGE}
41      container_name: hw5-composeprac
42      env_file:
43        - ./env/app.env # STUDENTID, HELPER_BASE_URL

```

```

43 - ./env/redis.env          # REDIS_HOST, REDIS_PORT
44 ports: ["${APP_PORT}:8080"]
45 networks: [edge-net, redis-net]
46 depends_on:
47     redis: { condition: service_healthy }
48     nginx: { condition: service_healthy }
49
50 networks:
51     edge-net: { driver: bridge }
52     redis-net: { driver: bridge, internal: true }
53     helper-net: { driver: bridge, internal: true }
54
55 volumes:
56     redis-data:

```

فایل‌های محیطی:

# env/app.env	# env/redis.env
STUDENTID=401105561	REDIS_HOST=redis
HELPER_BASE_URL=http://nginx:80	REDIS_PORT=6379

:nginx.conf

```

1 worker_processes auto;
2 events { worker_connections 1024; }
3
4 http {
5     include        /etc/nginx/mime.types;
6     default_type    application/json;
7     log_format proxy '$remote_addr -> $upstream_addr "$request" '
8                     'status=$status upstream_status=$upstream_status';
9     access_log /var/log/nginx/access.log proxy;
10    server_tokens off;
11
12    resolver 127.0.0.11 valid=10s ipv6=off;    # docker embedded DNS
13
14    upstream helper_backend {
15        server helper:9090;
16        keepalive 16;
17    }
18
19    server {
20        listen 80;
21        server_name _;
22
23        # the only endpoint the java app calls: GET ${HELPER_BASE_URL}/helper
24        location = /helper {
25            proxy_pass http://helper_backend/helper;
26            proxy_http_version 1.1;
27            proxy_set_header    Connection "";
28            proxy_set_header    Host $host;
29            proxy_set_header    X-Forwarded-For $proxy_add_x_forwarded_for;
30            proxy_connect_timeout 3s;
31            proxy_read_timeout 10s;
32        }
33
34        location = /healthz { access_log off; return 200 '{"status":"ok"}\n'; }
35        location / { return 404 '{"error":"not found"}\n'; }
36    }
37 }

```

سرویس کمکی هم با یک Dockerfile کوچک روی python:3.12-alpine که EXPOSE 9090 در آن فقط مستندسازی است و هیچ پورته روی میزبان باز نمی‌کند.

اجرا و دو خط لاگ:

```
1 docker build -t composeprac compose-challenge
2 docker compose up -d
3 curl -s -X POST http://localhost:8080/call-me
4 docker compose logs app
```

```
q6 — docker compose logs app

$ docker compose ps
NAME                STATUS              PORTS
hw5-composeprac     Up 15 seconds      0.0.0.0:8080->8080/tcp
hw5-helper          Up 26 seconds (healthy)
hw5-nginx           Up 20 seconds (healthy)  80/tcp
hw5-redis           Up 26 seconds (healthy)

$ curl -s -X POST http://localhost:8080/call-me
{"msg":"check the logs!"}

$ docker compose logs app --no-log-prefix
HELLO, Moeein!
FLAG={W3lcome_T0_the_P1@ygr0und}
```

شکل ۴: دو خط لاگ خواسته‌شده: پیام شخصی و فلگ.

دو خط لاگ به دست آمده:

```
HELLO, Moeein!
FLAG={W3lcome_T0_the_P1@ygr0und}
```

برای اینکه نشان دهیم nginx واقعا در مسیر است (و نه صرفا کنار آن)، از یک کانتینر روی edge-net یک بار مستقیم به helper و یک بار از طریق nginx درخواست زده شد:

```
q6 — network isolation

$ docker inspect ... --format "{{.Name}}: networks"
/hw5-composeprac: ms-hw5-compose_edge-net ms-hw5-compose_redis-net
/hw5-nginx: ms-hw5-compose_edge-net ms-hw5-compose_helper-net
/hw5-helper: ms-hw5-compose_helper-net
/hw5-redis: ms-hw5-compose_redis-net

# the java app shares no network with helper -> it cannot reach it directly
$ docker run --rm --network ms-hw5-compose_edge-net alpine wget -T3 -q0- http://helper:9090/helper
wget: bad address 'helper:9090'

# the same call through nginx succeeds
$ docker run --rm --network ms-hw5-compose_edge-net alpine wget -T3 -q0- http://nginx:80/helper
{"msg": "check the logs!"}

$ docker compose logs nginx --no-log-prefix | tail -3
172.23.0.3 -> 172.24.0.2:9090 "GET /helper HTTP/1.1" status=200 upstream_status=200 rt=0.001 urt=0.001
172.23.0.3 -> 172.24.0.2:9090 "GET /helper HTTP/1.1" status=200 upstream_status=200 rt=0.000 urt=0.000
172.23.0.4 -> 172.24.0.2:9090 "GET /helper HTTP/1.1" status=200 upstream_status=200 rt=0.000 urt=0.000
```

شکل ۵: از شبکه‌ی برنامه‌ی جاوا، نام helper اصلا قابل resolve نیست؛ اما همان درخواست از مسیر nginx پاسخ می‌گیرد و در access log انجینکس ثبت می‌شود.

استفاده‌ی داکر کامپوز از فایل‌های env

کامپوز دو سازوکار کاملاً متفاوت دارد که اغلب با هم اشتباه گرفته می‌شوند:

۱. فایل `env`. کنار `docker-compose.yml`. این فایل به‌صورت خودکار خوانده می‌شود و مقادیرش درون خود فایل کامپوز جای‌گذاری می‌شود؛ یعنی روی نام ایمج، شماره‌ی پورت، نام پروژه و هر جای دیگری که `{VAR}` نوشته شده اثر می‌گذارد. این متغیرها به‌خودی‌خود وارد کانتینر نمی‌شوند. در این تمرین `env` شامل `APP_IMAGE`، `APP_PORT`، `REDIS_IMAGE` و `NGINX_IMAGE` است.
 ۲. کلید `env_file` زیر یک سرویس. محتوای این فایل‌ها به‌عنوان متغیر محیطی داخل همان کانتینر تزریق می‌شود و در فایل کامپوز قابل استفاده نیست. اینجا `env/app.env` و `env/redis.env` دقیقاً همان چهار متغیر مورد نیاز برنامه‌ی جاوا را می‌دهند.
- نکات کاربردی:

- ترتیب اولویت از کم به زیاد: مقدار پیش‌فرض در `Dockerfile (ENV)` ← `env_file` ← `environment` ← متغیر پاس‌داده‌شده در خط فرمان.
- چند `env_file` مجاز است و در تعارض، فایل بعدی برنده است.
- فایل `env_file` با `shell` تفسیر نمی‌شود؛ نه بسط متغیر دارد، نه `export` می‌خواهد و کوتیشن‌ها بخشی از مقدار محسوب می‌شوند.
- مسیر `env`. پیش‌فرض کنار فایل کامپوز است و با `env-file` قابل تعویض است؛ الگوی رایج داشتن `env.example`. در مخزن و نگه داشتن `env`. واقعی بیرون از گیت است.

انواع Network در داکر

توضیح	دراپور
شبکه‌ی خصوصی روی یک میزبان. شبکه‌ی <code>bridge</code> کاربرساخته DNS داخلی دارد و کانتینرها با نام یکدیگر را می‌بینند (برخلاف <code>bridge</code> پیش‌فرض).	bridge
کانتینر مستقیماً از <code>network namespace</code> میزبان استفاده می‌کند؛ جداسازی شبکه ندارد و <code>port mapping</code> بی‌معناست. برای کارایی بالا یا کشف سرویس در سطح میزبان.	host
هیچ رابط شبکه‌ای جز <code>loopback</code> ؛ برای کارهای کاملاً ایزوله و پردازش داده‌ی نامطمئن.	none
شبکه‌ی چندمیزبانه روی <code>Swarm</code> ؛ کانتینرهای ماشین‌های مختلف را در یک L2 مجازی قرار می‌دهد و رمزنگاری اختیاری دارد.	overlay
به کانتینر آدرس MAC و IP مستقل روی شبکه‌ی فیزیکی می‌دهد؛ برای سامانه‌های قدیمی که باید مستقیماً در شبکه‌ی سازمان دیده شوند.	macvlan
مشابه <code>macvlan</code> اما با اشتراک MAC؛ مناسب جایی که سویچ محدودیت MAC دارد.	ipvlan

دو صفت مهم و مستقل از دراپور:

- `internal: true`: شبکه به دنیای بیرون مسیر ندارد. در این تمرین `redis-net` و `helper-net` از این نوع‌اند؛ یعنی حتی اگر کسی در آن کانتینرها کد دلخواه اجرا کند، خروجی به اینترنت ندارد.
- `external: true`: شبکه از قبل بیرون از کامپوز ساخته شده و مدیریتش با کامپوز نیست (توضیح سؤال ۱).

پاسخ مسئله‌ی ۷. پایپلاین‌طور

یک سرویس HTTP به زبان Go 1.23 با دو API GET /health و POST /v1/analyze (آمار واژگان یک متن) نوشته شد؛ ۱۵ تست واحد با پوشش ۷۰ درصدی کل و ۱۰۰ درصدی بسته‌ی منطق دامنه. تنها وابستگی بیرونی `github.com/google/uuid` است تا مرحله‌ی `install` کار واقعی داشته باشد. سرویس `graceful shutdown` دارد و شماره‌ی نسخه‌اش در زمان بیلد با `-ldflags` روی باینری حک می‌شود؛ همین باعث می‌شود در مرحله‌ی `deploy` بتوان تأیید کرد که دقیقاً همان `artifact` این پایپلاین اجرا شده است.

چون `GitLab Runner` نداریم، اسکریپت `runner.bash` نقش آن را بازی می‌کند. هر جاب دقیقاً مثل `docker executor` در یک کانٹینر یک‌بار مصرف اجرا می‌شود:

```
1 in_docker() {
2     local image="$1"; shift
3     docker run --rm \
4         -v "$WORKSPACE:/src" \
5         -v "$ARTIFACTS_DIR:/artifacts" \
6         -v "$MOD_CACHE_VOL:/go/pkg/mod" \
7         -v "$BUILD_CACHE_VOL:/root/.cache/go-build" \
8         -w /src -e CGO_ENABLED=0 -e GOFLAGS=-buildvcs=false \
9         "$image" sh -eu -c "$*"
10 }
```

دو جایی که باید خود `daemon` را صدا بزنند (`build` و `deploy`) نسخه‌ی دومی از همین تابع را استفاده می‌کنند که `/var/run/docker.sock` را هم `mount` می‌کند — همان الگویی که رانرهای واقعی برای ساخت ایمپج به کار می‌برند.

جاب	ایمپج	کار و شرط عبور
install	golang:1.23-alpine	go mod verify و go mod download
lint	+ golang:1.23-alpine golangci-lint:v1.62.2	go vet، gofmt -l، سپس golangci-lint (revive، staticcheck، errcheck و ...)
build	docker:27-cli + golang	باینری ایستا به‌عنوان artifact، سپس ساخت ایمپج
test	golang:1.23-alpine	go test -race با گزارش پوشش
deploy	alpine + docker:27-cli	استقرار ماک و smoke test

`deploy` به این صورت ماک است که یک شبکه‌ی موقت می‌سازد، ایمپج همین پایپلاین را روی آن بالا می‌آورد، از یک کانٹینر دیگر روی همان شبکه `/health` و `/v1/analyze` را صدا می‌زند و در پایان چه موفق و چه ناموفق با یک `trap` محیط را آزاد می‌کند. یعنی چیزی واقعا اجرا و آزموده می‌شود، ولی هیچ محیط پایداری درگیر نمی‌شود.

امکاناتی که از `GitLab CI` تقلید شده‌اند:

- کش: دو `volume` به نام‌های `q7-go-build-cache` و `q7-go-mod-cache` بین اجراها باقی می‌مانند (معادل `cache`).
- `artifact`: باینری، `coverage.out` و `coverage.html` در `.ci/artifacts/` و لاگ جداگانه‌ی هر جاب در `.ci/logs/`.
- `dotenv` بین جاب‌ها: جاب `build` تگ ایمپجی که ساخته را در `build.env` می‌نویسد و `deploy` همان را می‌خواند؛ معادل `artifacts:reports:dotenv`. بدون این، `deploy` نمی‌داند کدام ایمپج را مستقر کند.
- `fail-fast`: اولین جاب شکست‌خورده باعث می‌شود بقیه `skipped` علامت بخورند و کد خروج اسکریپت غیرصفر شود.
- `allow_failure`: فهرستی از جاب‌ها که شکستشان پایپلاین را متوقف نمی‌کند (فعلا خالی).
- اجرای گزینشی: `runner.bash lint test`. فقط همان جاب‌ها را اجرا می‌کند.

نتیجه‌ی اجرا در حالت موفق:

```

q7 -- ./runner.bash

pipeline workspace=/Users/moein/Documents/SUT-Courses/CE222-MS/Assignments/Assignment05/solutions/Q7-pipeline/app tag=20260806-134032
stages: install lint build test deploy
==> install
all modules verified
--- modules ---
example.com/ci-demo
github.com/google/uuid v1.6.0
PASS install (0s) log: .ci/logs/install.log
==> lint
gofmt: clean
go vet: clean
golangci-lint: clean
PASS lint (3s) log: .ci/logs/lint.log
==> build
-rwxr-xr-x 1 root root 4.9M Aug 6 10:10 /artifacts/api
image [ci-demo:20260806-134032 ci-demo:latest] size=14003241 bytes
artifacts: api, build.env (APP_TAG=20260806-134032)
PASS build (2s) log: .ci/logs/build.log
==> test
example.com/ci-demo/cmd/api coverage: 0.0% of statements
ok example.com/ci-demo/internal/httpapi 1.020s coverage: 87.5% of statements
ok example.com/ci-demo/internal/textstats 1.029s coverage: 100.0% of statements
--- coverage ---
example.com/ci-demo/internal/httpapi/handler.go:63: writeJSON 75.0%
example.com/ci-demo/internal/textstats/textstats.go:25: Tokenize 100.0%
example.com/ci-demo/internal/textstats/textstats.go:38: Analyze 100.0%
total: (statements) 70.5%
PASS test (50s) log: .ci/logs/test.log
==> deploy
[deploy] deploying ci-demo:20260806-134032 (from build.env)
[deploy] container started, waiting for readiness...
[deploy] GET /health -> {"status":"ok","version":"20260806-134032"}
[deploy] POST /v1/analyze -> {"request_id":"780088ca-3a77-43c6-9bdd-0a8ca83712d9","summary":{"words":4,"unique":3,"top":{"word":"deploy","count":2},"word":"smoke","count":1}}}
[deploy] smoke tests passed, releasing the mock environment
PASS deploy (1s) log: .ci/logs/deploy.log
pipeline summary
-----
ok install 0s
ok lint 3s
ok build 2s
ok test 50s
ok deploy 1s
-----
total: 56s
PIPELINE PASSED

```

شکل ۶: اجرای کامل `./runner.bash`؛ همه‌ی پنج جاب سبز و کد خروج صفر.

نتیجه‌ی اجرا در حالت شکست:

برای اینکه نشان دهیم هر دروازه واقعا مسدود می‌کند، اسکریپت `demo-failure.bash` پروژه را در یک پوشه‌ی موقت کپی می‌کند، یک نقص واقعی تزریق می‌کند و پایپالین را روی همان کپی اجرا می‌کند (سورس اصلی دست‌نخورده می‌ماند):

مرحله	نقص تزریق شده
lint	فایلی با قالب‌بندی نادرست و یک خطای بررسی نشده
build	COPY از فایلی که وجود ندارد در Dockerfile
test	تغییر یک انتظار در تست واحد
deploy	LISTEN_ADDR نامعتبر، پس سرویس هرگز سالم نمی‌شود

```

q7 -- ./demo-failure.bash

$ ./demo-failure.bash lint      # injected defect: gofmt/errcheck findings in internal/textstats/broken.go
FAIL lint (0s) log: .ci/logs/lint.log
pipeline summary
-----
ok      install    1s
failed lint      0s
skipped build    -
skipped test     -
skipped deploy   -
-----
total: 1s
PIPELINE FAILED

$ ./demo-failure.bash build      # injected defect: Dockerfile COPY of a file that does not exist
FAIL build (1s) log: .ci/logs/build.log
pipeline summary
-----
ok      install    0s
ok      lint      4s
failed build  1s
skipped test  -
skipped deploy -
-----
total: 5s
PIPELINE FAILED

$ ./demo-failure.bash test      # injected defect: a unit-test expectation is violated
FAIL test (54s) log: .ci/logs/test.log
pipeline summary
-----
ok      install    0s
ok      lint      3s
ok      build      1s
failed test  54s
skipped deploy -
-----
total: 58s
PIPELINE FAILED

$ ./demo-failure.bash deploy      # injected defect: the container never becomes healthy (bad LISTEN_ADDR)
FAIL deploy (17s) log: .ci/logs/deploy.log
pipeline summary
-----
ok      install    0s
ok      lint      2s
ok      build      2s
ok      test       61s
failed deploy  17s
-----
total: 82s
PIPELINE FAILED

```

شکل ۷: در هر چهار حالت، جاب معیوب failed می‌شود، جاب‌های بعدی skipped می‌مانند و پایپلاین با کد غیرصفر خاتمه می‌یابد.

پاسخ مسئله‌ی ۸. نکسوس

کل پیکربندی به‌جای کلیک در رابط گرافیکی، از طریق REST API نکسوس و در قالب اسکریپت انجام شده تا تکرارپذیر و قابل بازبینی باشد.

الف) بالا آوردن نکسوس

```

1 services:
2   nexus:
3     image: sonatype/nexus3:3.79.1
4     container_name: hw5-nexus
5     environment:
6       INSTALL4J_ADD_VM_PARAMS: "-Xms1g -Xmx2g -XX:MaxDirectMemorySize=2g"
7     ports:
8       - "8081:8081" # UI + REST API + pypi/npm repository endpoints
9       - "8082:8082" # HTTP connector of the hosted docker registry
10    volumes: [nexus-data:/nexus-data]
11    ulimits: { nofile: { soft: 65536, hard: 65536 } }
12    healthcheck:
13      test: ["CMD", "sh", "-c", "curl -sf http://localhost:8081/service/rest/v1/
14      status || exit 1"]
15      interval: 15s
16      retries: 40
17      start_period: 60s

```

نکته: ایمج sonatype/nexus3 تنها از نسخه‌ی 3.78 به بعد arm64 منتشر می‌کند؛ تگ‌های قدیمی‌تر روی پردازنده‌های Apple Silicon با شبیه‌سازی x86 اجرا می‌شوند که به‌شدت کند است.

ب) مراحل انجام‌شده

۱. رمز اولیه: نکسوس رمز تصادفی را در `/nexus-data/admin.password` می‌گذارد؛ اسکریپت آن را می‌خواند و با یک درخواست PUT تعویض می‌کند.
۲. پذیرش EULA: نسخه‌های Community Edition از 3.78 به بعد تا وقتی توافق‌نامه پذیرفته نشود به هر داندلود artifact پاسخ 403 می‌دهند. نکته‌ی گمراه‌کننده این است که فهرست `/simple/<pkg>/` با کد ۲۰۰ برمی‌گردد و فقط دریافت خود فایل رد می‌شود؛ خطا شبیه مشکل دسترسی به نظر می‌رسد ولی نیست.
۳. Blob File: ساخت `hw5-blob` از نوع File.
۴. مخازن Proxy: `pypi-proxy` به `https://pypi.org/` و `npm-proxy` به `https://registry.npmjs.org`، هر دو با storage روی `hw5-blob`.
۵. مخزن Group: `pypi-group` شامل `pypi-proxy`.
۶. realm: فعال کردن `DockerToken`.
۷. مخزن داکر: `docker-hosted` با connector روی پورت ۸۰۸۲ و `writePolicy: ALLOW`.

```

1 create "pypi-proxy" /service/rest/v1/repositories/pypi/proxy '{
2   "name": "pypi-proxy",
3   "online": true,
4   "storage": { "blobStoreName": "hw5-blob", "strictContentTypeValidation": true },
5   "proxy": { "remoteUrl": "https://pypi.org/", "contentMaxAge": 1440, "
   metadataMaxAge": 1440 },

```

```

6  "negativeCache": { "enabled": true, "timeToLive": 1440 },
7  "httpClient": { "blocked": false, "autoBlock": true }
8  }'

```

```

q8 ~. ./setup-nexus.sh

==> waiting for Nexus at http://localhost:8081
Nexus is available after 10s (roughly)

==> admin credentials
password already rotated to 'admin123'

==> End User License Agreement
already accepted

==> blob store
exists   hw5-blob (400 - already present)

==> proxy repositories (stored on hw5-blob)
exists   pypi-proxy -> https://pypi.org/ (400 - already present)
exists   npm-proxy -> https://registry.npmjs.org (400 - already present)

==> group repository (one URL in front of several repositories)
exists   pypi-group [pypi-proxy] (400 - already present)

==> enabling the Docker Bearer Token realm
realms/active -> http 204

==> hosted docker registry on port 8082
exists   docker-hosted (400 - already present)

==> result
NAME      FORMAT  URL
maven-releases  maven2  http://localhost:8081/repository/maven-releases
maven-snapshots maven2  http://localhost:8081/repository/maven-snapshots
maven-central   maven2  http://localhost:8081/repository/maven-central
maven-public    maven2  http://localhost:8081/repository/maven-public
nuget-hosted     nuget   http://localhost:8081/repository/nuget-hosted
nuget.org-proxy  nuget   http://localhost:8081/repository/nuget.org-proxy
nuget-group      nuget   http://localhost:8081/repository/nuget-group
pypi-proxy       pypi    http://localhost:8081/repository/pypi-proxy
npm-proxy        npm      http://localhost:8081/repository/npm-proxy
pypi-group       pypi    http://localhost:8081/repository/pypi-group
docker-hosted    docker  http://localhost:8081/repository/docker-hosted

==> blob stores
default (File)
hw5-blob (File)

```

شکل ۸: خروجی setup-nexus.sh؛ اسکریپت idempotent است و اجرای دوباره‌اش چیزی را خراب نمی‌کند.

ج) دریافت وابستگی‌ها از مخزن ساخته‌شده

پروژه‌ی سؤال ۴ برای این کار استفاده شد؛ چون همان‌جا سازوکار MIRROR_URL پیاده شده بود، تنها کاری که لازم بود دادن نشانی نکسوس به عنوان index است:

```

1 docker build \
2   --build-arg MIRROR_URL=http://host.docker.internal:8081/repository/pypi-proxy/
   simple \
3   --build-arg MIRROR_TRUSTED_HOST=host.docker.internal:8081 \
4   --target deps -t linkshrink:from-nexus ../Q4-dockerize

```

د) پوش و پول ایمج

```

1 docker login localhost:8082 -u admin

```

```

2 docker tag alpine:3.20 localhost:8082/hw5/alpine:3.20
3 docker push localhost:8082/hw5/alpine:3.20
4 docker rmi localhost:8082/hw5/alpine:3.20 # remove the local copy
5 docker pull localhost:8082/hw5/alpine:3.20 # pull it back from Nexus

```

```

q8 — ./verify-nexus.sh

==> A) building the Q4 image with pip pointed at pypi-proxy
index-url: http://host.docker.internal:8081/repository/pypi-proxy/simple
#11 0.462 Looking in indexes: http://host.docker.internal:8081/repository/pypi-proxy/simple
#11 0.516 Downloading http://host.docker.internal:8081/repository/pypi-proxy/packages/pip/26.2.1/pip-26.2.1-py3-none-any.whl (1.8 MB)
#11 1.086 Successfully installed pip-26.2.1
#11 1.437 Looking in indexes: http://host.docker.internal:8081/repository/pypi-proxy/simple
#11 1.474 Downloading fastapi-0.115.6-py3-none-any.whl (94 kB)
#11 1.495 Downloading uvicorn-0.34.0-py3-none-any.whl (62 kB)
#11 1.604 Downloading pydantic-2.10.4-py3-none-any.whl (431 kB)
#11 1.642 Downloading starlette-0.41.3-py3-none-any.whl (73 kB)

==> packages now cached in the pypi-proxy blob store
annotated-types 0.8.0
anyio 4.14.2
click 8.4.2
fastapi 0.115.6
h11 0.16.0
httpx 0.27.0
httpxtools 0.8.0
idna 3.18
pip 26.2.1
pydantic 2.10.4
pydantic_core 2.27.2
python-dotenv 1.2.2
PyYAML 6.0.3

==> B) npm-proxy metadata check
react latest = 19.2.8

==> C) docker login localhost:8082
Login Succeeded

==> tag + push
The push refers to repository [localhost:8082/hw5/alpine]
88b4fba61c4c: Preparing
88b4fba61c4c: Pushed
3.20: digest: sha256:d10bea758e065a0cbf1f2d524b90b30a2ef986bdb4294fe9dbdb5fa59174b068 size: 527

==> remove the local copy, then pull it back from Nexus
3.20: Pulling from hw5/alpine
Digest: sha256:d10bea758e065a0cbf1f2d524b90b30a2ef986bdb4294fe9dbdb5fa59174b068
Status: Downloaded newer image for localhost:8082/hw5/alpine:3.20
localhost:8082/hw5/alpine:3.20
alpine release: 3.20.10

==> component list of docker-hosted
hw5/alpine:3.20 (docker)

==> done

```

شکل ۹: خروجی verify-nexus.sh: نصب وابستگی‌های پایتون از pypi-proxy و کش شدن آن‌ها در blob store، پاسخ npm-proxy، و چرخه‌ی کامل login/push/pull روی docker-hosted.

یک نکته‌ی عملی که در مسیر به آن برخوردیم: با تنظیم پیش فرض (forceBasicAuth: false) دستور docker login خطای "null" unsupported protocol scheme شکست می‌خورد. علتش این است که جریان bearer token endpoint را از قابلیت Base URL نکسوس می‌سازد که در نصب تازه خالی است و از طریق REST API عمومی هم قابل تنظیم نیست. راه‌حل، یا تنظیم Base URL در رابط گرافیکی است یا فعال کردن forceBasicAuth روی مخزن تا رجیستری با HTTP Basic احراز هویت کند؛ راه دوم انتخاب شد چون کاملاً در اسکریپت قابل انجام است.

ه) نقش‌های نکسوس در یک پایپلاین CI/CD

۱. کش و میروور وابستگی‌ها: مخازن proxy باعث می‌شوند بیلد به در دسترس بودن pypi.org یا npmjs.com

وابسته نباشد. علاوه بر سرعت و کاهش ترافیک، در محیط‌هایی که دسترسی به اینترنت محدود یا فیلتر است، عملاً تنها راه ادامه‌ی کار همین است.

۲. مخزن artifact‌های خودی: خروجی بیلد (jar, wheel)، پکیج (npm داخلی) در مخازن hosted منتشر می‌شود و سرویس‌های دیگر با شماره‌ی نسخه آن را مصرف می‌کنند.

۳. رجیستری ایمیج: همان نقشی که در این تمرین دیدیم؛ نقطه‌ی تحویل بین CI و CD.

۴. تفکیک نسخه‌های ناپایدار از پایدار: جدا کردن snapshot و release در دو مخزن و اجازه‌ی promote از اولی به دومی، به جای بیلد دوباره.

۵. کنترل دسترسی و ممیزی: نقش و کاربر جداگانه برای CI (فقط خواندن از proxy ها و نوشتن در hosted)، به همراه لاگ اینکه چه کسی چه چیزی را چه زمانی منتشر کرده است.

۶. تکرارپذیری و امنیت زنجیره‌ی تأمین: چون artifact کش شده در نکسوس می‌ماند، حتی اگر نسخه‌ای از مخزن عمومی حذف شود بیلدهای قدیمی همچنان قابل بازتولیدند. همچنین می‌توان با قواعد Content Selector و Routing Rule جلوی ورود بسته‌های خاص یا آسیب‌پذیر را گرفت.

۷. مدیریت چرخه‌ی عمر: سیاست‌های cleanup برای حذف خودکار snapshot‌های قدیمی و ایمیج‌های بدون استفاده تا فضای دیسک کنترل شود.

و) کاربرد مخزن نوع Group

مخزن Group خودش چیزی ذخیره نمی‌کند؛ چند مخزن هم‌فرمت را زیر یک نشانی واحد جمع می‌کند و درخواست را به ترتیب تعریف شده بین اعضا جست‌وجو می‌کند و اولین پاسخ را برمی‌گرداند.

فایده‌ی اصلی‌اش این است که کلاینت فقط یک نشانی می‌شناسد. برای مثال به جای اینکه در pip.conf هر توسعه‌دهنده هم نشانی مخزن داخلی و هم نشانی proxy نوشته شود، یک pypi-group تعریف می‌کنیم که شامل pypi-hosted (بسته‌های خودمان) و pypi-proxy (بسته‌های عمومی) است. نتیجه:

- پیکربندی سمت کلاینت ساده و ثابت می‌ماند؛ افزودن یک مخزن جدید فقط یک تغییر سمت سرور است و هیچ توسعه‌دهنده‌ای لازم نیست تنظیماتش را عوض کند.
- ترتیب اعضا معنادار است: قرار دادن مخزن داخلی قبل از proxy باعث می‌شود بسته‌ی هم‌نام داخلی برنده شود؛ این هم برای اولویت‌دهی به کد خودی مفید است و هم یک لایه‌ی دفاعی در برابر حمله‌ی dependency confusion است.
- در Group‌های داکر، همین ویژگی اجازه می‌دهد یک endpoint واحد هم ایمیج‌های داخلی و هم ایمیج‌های Docker Hub را سرو کند.
- محدودیت مهم: Group فقط برای خواندن است؛ برای انتشار باید مستقیماً روی مخزن hosted پوش کرد.